

Геометрический детектор препятствий в тоннеле метро

Команда: MISIS_ — Давыдов Денис, Кулагин Александр, Голышев Юрий

Задача: обнаружение посторонних объектов перед поездом по данным 3D-лидара

О команде

- **Команда:** MISIS_
- **Участников:** 3 человека

О РЕШЕНИИ И КОМАНДЕ

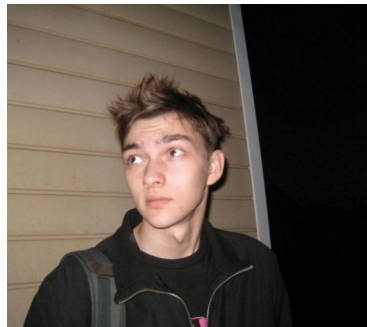
Краткое описание решения:

Не классифицируем объекты, а описываем нормальный тоннель — рельсы, стены, габарит — и ищем всё, что в него не вписывается. Без обучения на размеченных препятствиях.

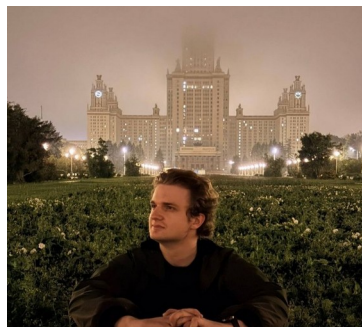
Уникальность решения:

Ось пути ведём по стенам там, где рельсы уже не видны (коридор измерен до 77–175 м); границу габарита задают постоянные конструкции; скорость поезда оценивается без одометрии.

КОМАНДА



Голышев Юрий
МИСИС



Кулагин Александр
МИСИС



Давыдов Денис
НИУ ВШЭ

Почему вы выбрали именно эту задачу из предложенных на хакатоне?

01

В тоннеле почти всегда пусто, размеченных препятствий мало. Идея из ТЗ — хорошо описать нормальный тоннель и искать всё, что в него не вписывается, — показалась нам самой честной.

С какими основными сложностями или вызовами вы столкнулись и как их преодолели?

02

Реальные записи показали то, чего не было на симуляторе: уклон пути и лоток, служебный проход, неровные стены, кривые, гермозатворы. Первая версия давала 14 ложных кадров из 1611 — после доработок остался 1.

Техническая суть решения

- Вход: облако Hesai-128 (~300 тыс. точек), ROS 2 Humble, Docker
- Ось пути: рельсы вблизи (40–70 м), дальше по стенам — до 77–175 м
- Модель нормального сечения пути: уклон, лоток, полки
- Габарит по постоянным конструкциям, DBSCAN, трекер 2/3, 3/5, 4/6
- Выход: расстояние, категория, уверенность, TTC, флаг торможения

Маркетинговая суть решения

- Решение обходится без размеченных препятствий и работает с любым тоннелем, который можно описать геометрией
- Запуск для инженера метро: docker build → docker run → ros2 bag play
- Дальше: геометрия пути в отдельный поток или C++, проверка дальности на новых записях, безопасная экстраполяция оси в кривых

01

Тоннель почти всегда пуст

- Размеченных примеров препятствий мало, обучать классификатор объектов не на чем.

02

Однородная среда

- Рельсы, стены, кабели и одинаковые конструкции. Важно не «найти человека», а понять, что не вписывается в норму.

03

Луч скользит вдоль пути

- Рельсы пропадают из облака на 40–70 м, а препятствие нужно увидеть заранее. Дальше ось пути приходится вести по стенам.

01

Ось пути

- Рельсы вблизи, стены тоннеля вдали: коридор измерен до 77–175 м.

02

Сечение пути

- Модель нормального профиля: уклон, лоток, полки. Они не дают ложных «низких объектов».

03

Габарит

- Границу задают постоянные конструкции, мимо которых поезд уже проходит.

04

Трекинг

- 3 кадра с компенсацией движения и трекер: 2/3, 3/5 или 4/6 кадров по дальности.

• Образец текста

- Предобработка
- Очистка, перевод в систему поезда

1

• Образец текста

- Ось пути
- Рельсы вблизи, стены вдали

3

• Образец текста

- Норма тоннеля
- Сечение пути и габарит

5

• Образец текста

- Кластеризация
- 3 кадра, DBSCAN с растущим eps

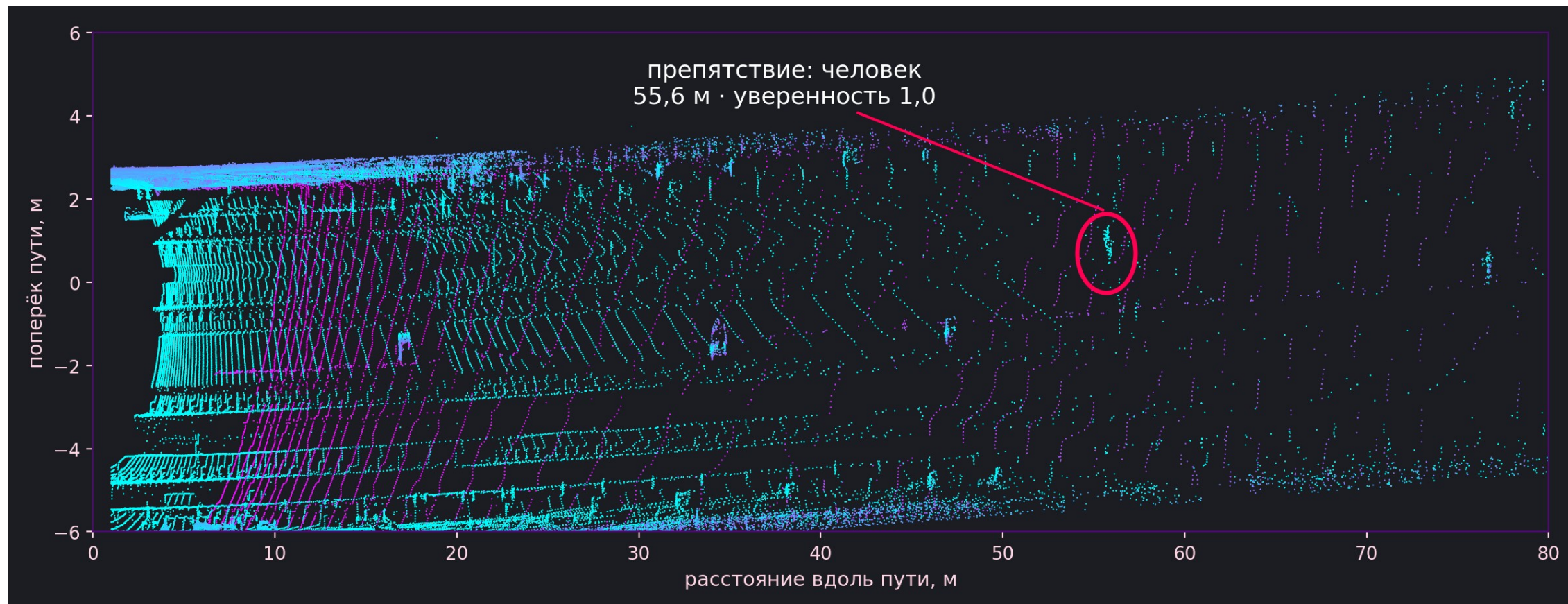
2

• Образец текста

- Результат
- Расстояние, TTC, флаг торможения

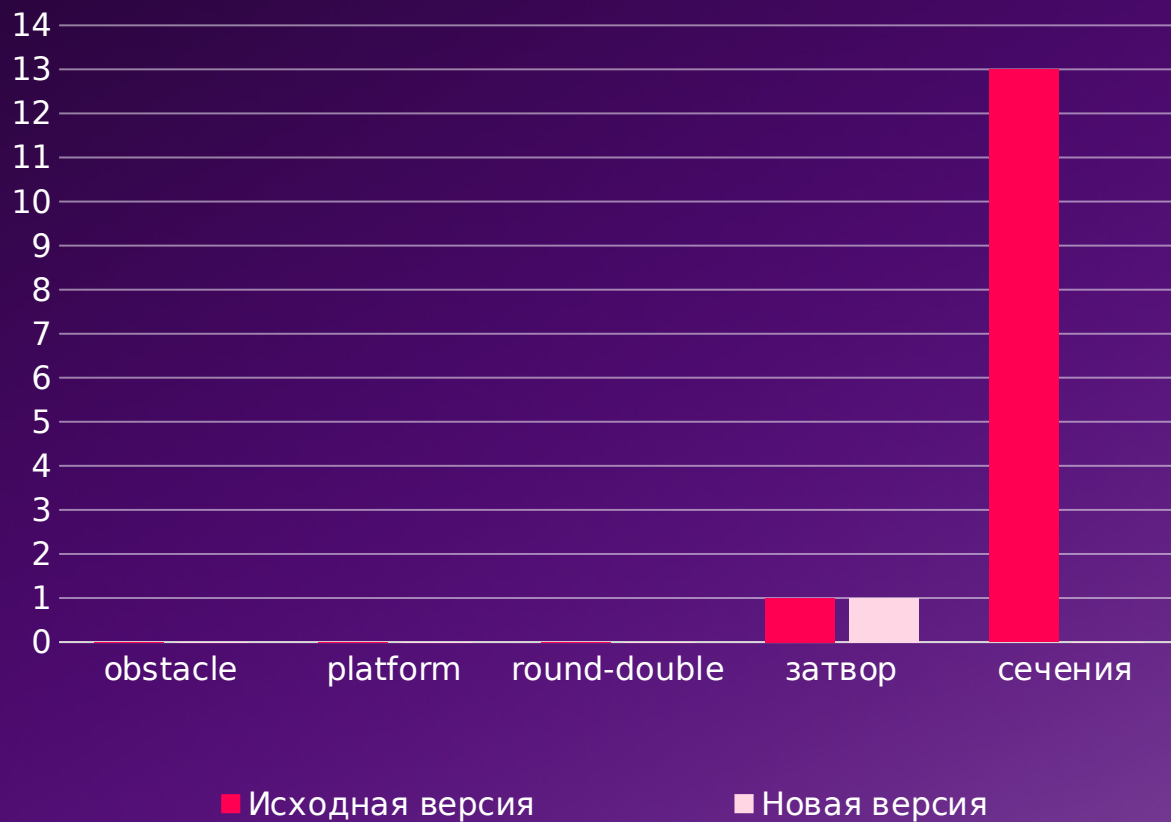
4

ОТ ОБЛАКА К ПРЕПЯТСТВИЮ



Запись doubleT_obstacle: человек пересекает путь на 55-57 м. Первое подтверждение — кадр 16, 55,6 м, уверенность 1,0; ложных кадров в записи 0.

РЕЗУЛЬТАТЫ



14 → 1

ложных кадров из 1611 на реальных записях

47 кадров

с человеком на 55-57 м, первое подтверждение на 16-м кадре

≈ 100-115 мс

на кадр полного облака (медиана, p95 ≈ 150 мс) на 2 слабых ядрах облака

Симулятор: человек, поезд стоит

на 30 м — 100 % кадров

на 60 м — 100 %

на 100 м — 56 %

на 150 м — 0 %

В реальных записях препятствий дальше 57 м нет, поэтому дальность проверена только на симуляторе.

Уклон пути и лоток → модель сечения пути

Служебный проход в 1,1-1,4 м → габарит по конструкциям

Неровные стены → медиана по стене, калибровка по сплошной стороне

Переходная кривая → экстраполяция оси исключена

Платформа «подхватывала» пол → ограничения уровня пола

ЗАПУСК И ВЫХОД



Запуск

`docker build → docker run →
ros2 launch demo.launch.py.`
Bag читается напрямую, все
параметры — ROS params.

Выход

`/detected, /distance,
/confidence,
/brake_required, /status`
(JSON) и маркеры для RViz.

Проверка

15 тестов, включая end-to-
end на симуляторе; утилиты
для прогона записей без
ROS.

ОГРАНИЧЕНИЯ И ПЛАНЫ

- **Границы метода**
- Дальность больше 57 м на реальных данных не проверена: в записях нет таких препятствий.
- На симуляторе 100 м — 56 % кадров, 150 м — 0 %. Пороги подобраны на тех же записях: отложенной выборки нет.
- **Планы**
- Проверить на отложенных записях, вынести геометрию пути в отдельный поток или C++, вернуть безопасную экстраполяцию оси в кривых.

57 м

дальность, подтверждённая на реальных записях: человек на 55–57 м

100 м

на симуляторе: человек обнаруживается в 56 % кадров, на 150 м — уже нет